2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA) | 979-8-3503-2658-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISCA59077.2024.00076

Soter: Analytical Tensor-Architecture Modeling and Automatic Tensor Program Tuning for Spatial Accelerators

Fuyu Wang
Sun Yat-Sen University
Guangzhou, China
fuyuwang17@gmail.com

Minghua Shen†
Sun Yat-Sen University
Guangzhou, China
shenmh6@mail.sysu.edu.cn

Yufei Ding
University of California, San Diego
California, USA
yufeiding@ucsd.edu

Nong Xiao†
Sun Yat-Sen University
Guangzhou, China
xiaon6@mail.sysu.edu.cn

***Abstract*—Spatial accelerator is a specialized hardware to provide noticeable performance speedup for tensor computations. It also brings a challenge to map tensor computations on spatial accelerators. Auto-tuning compiler is one of the most promising directions for tensor mapping. However, existing auto-tuning compilers suffer from either numerous invalid and inefficient programs or inaccurate evaluation of incomplete programs, leading to sub-optimal performance.**

In this paper, we propose Soter, a novel auto-tuning tensor compilation framework for spatial accelerators. The key is to perform exploration in a both valid and efficient program design space and perform optimization according to accurate evaluation of complete programs. First, we design an analytical model to generate a high-quality program design space, which excludes invalid and inefficient programs. Second, we design an automatic program tuner to efficiently explore the program space and avoid evaluating incomplete programs. Finally, we coordinate the model and the tuner to further improve the quality of program space. The program space is identified by the model and is updated during the exploration of tuner. On average, Soter achieves 2.1× to 3.5× speedup over the state-of-the-art tensor compilers. Moreover, Soter shows better scalability for larger-scale tensor computations and spatial architectures.

I. Introduction

Tensor computations play a crucial role in various domains such as scientific computing [1]–[3], social network [4]–[6], and deep learning [7]–[9]. Hardware specialization [10] has emerged as a potential direction to accelerate tensor computations. As one of the recent advancements, spatial accelerators [11]–[20] organize a large array of processing elements (PEs) and a multi-level memory hierarchy. By aligning tensor computations with memory access patterns in nested loops, spatial accelerators can obtain better performance speedup for tensor computations, compared with general-purpose processors such as CPUs and GPUs. The primary challenge faced by spatial accelerators is tensor mapping, which delivers the computation and data to PEs and memory hierarchy, respectively. This process requires to exploit parallelism among PEs and data reuse in memory hierarchy at compile time [21]–[27].

Tensor mapping techniques can be broadly categorized into three categories: vendor library, polyhedral model, and auto-tuning. Deep learning frameworks [28]–[30] map the operators in tensor computations to vendor-provided manually-optimized libraries [31], [32]. These libraries require huge engineering efforts and long development cycles, hindering the evolution of operators and accelerators. The polyhedral compilers [26], [33]–[35] formulate the tensor program tuning as a constrained-optimization problem and use integer linear programming (ILP) to solve it. These compilers rely on hand-tuned heuristics to co-optimize loop tiling with other loop transformations [26], [27]. They generally provide inferior performance for accelerators compared to auto-tuning compilers [27] and face a scalability challenge with ILP [36]. We focus on the auto-tuning in this paper. The auto-tuning compilers [24], [37]–[43] perform automatic tuning in the tensor program design space to generate an optimized program. They leverage a broad range of technologies, including beam search [44], simulated annealing [45], and evolutionary search [46]. Recent efforts focus on analytical cost models [21], [22], [47] and deep learning-based cost models [25], [38], [48] to guide the tuning process efficiently.

While auto-tuning compilers have reduced manual efforts and achieved great success, they still face the following challenges. We divide the existing auto-tuning compilers into two categories: sequence-guided and template-guided. The sequence-guided compilers [49] evaluate all choices of each tunable parameter (e.g., tile factor and parallel factor) across incomplete programs and select the top-k candidates. The incomplete program only contains partial parameters. Unfortunately, evaluating incomplete programs is inaccurate [25] and leads to cumulative errors during sequential decisions. Increasing the beam size k can tolerate inaccurate estimation but results in longer compile times. The template-guided compilers [24], [25], [37], [38] simultaneously determine all of the parameters to perform optimization across complete tensor programs. However, simultaneous decision-making prevents the compilers from accurately constraining the sub-space of each parameter. After exploring new parameters, they cannot update the sub-spaces of other parameters. Thus these compilers encounter a large number of invalid and inefficient program candidates during the tuning process. The invalid programs

† Corresponding Authors

979-8-3503-2658-1/24/$31.00 ©2024 IEEE
DOI 10.1109/ISCA59077.2024.00076

violate architectural constraints while the inefficient ones lead to low resource utilization of spatial accelerators. Both of them decrease the program tuning efficiency of compilers.

To address these challenges, we first propose an analytical tensor-architecture model to exclude invalid and inefficient programs. The insight is that we can build a set of inequalities between tunable parameters and architectural constraints. For example, the tile factors (X) are constrained by buffer capacity (C) since the tiled tensor is stored in the memory hierarchy. The example inequality is $f(X) \leq C$, where $f(X)$ represents the size of tiled tensor (the required buffer capacity). We can build other inequalities by analyzing the computation and data movement of tensors on the spatial architectures similarly. For each tunable parameter, the analytical model constrains its sub-space by fixing other parameters and solving the resulting inequalities. The sub-space is defined with maximum and minimum values. In this way, the constrained sub-spaces form a high-quality program design space.

Then, we propose an automatic tensor program tuner to converge to high-performance solutions. This is based on the following insights. 1) The tuner determines tunable parameters through a sequence of decisions. When exploring a certain parameter within its sub-space, it can fix the determined parameters and set the undetermined parameters to a default value (e.g., 1). After determining a parameter, the analytical model can bring the choice of this parameter into the inequalities to update the sub-spaces of undetermined parameters. The sub-space of each parameter is always defined with its maximum and minimum. In this way, we can coordinate the analytical model and the automatic tuner to optimize both tensor programs and program space. 2) The tuner exploits the Transformer [8] structure due to its robust ability in sequence modeling. Each tensor program is regarded as a language sequence while the tunable parameters are regarded as language tokens. When determining a specific parameter at each position, the Transformer model first produces a probability distribution over the sub-space of the parameter. For any invalid parameter choice, the corresponding probability is set to 0. Consequently, the Transformer model guarantees the validity of the generated tensor program. 3) Combining Transformer with the policy gradient algorithm [50], the tuner can select the best exploration direction instead of top-k candidates. Transformer is advantageous in scenarios characterized by sparse rewards [51], [52]. In this way, we only require to evaluate complete programs.

We integrate the analytical model and the automatic tuner into a tensor compilation framework called Soter. We evaluate Soter for representative spatial accelerators with various tensor computations (i.e., single operators and end-to-end workloads). We compare Soter with the state-of-the-art automatic paradigms, including auto-tuning compilers (i.e., Ansor [25] and Mind Mappings [38]) and a polyhedral compiler (i.e., CoSA [26]). On average, Soter achieves 2.1× to 3.3× speedup for Eyeriss [18]; 2.4× to 3.1× speedup for Simba [12]; 2.5× to 3.5× speedup for TensorCore [17]; 2.9× speedup for a specifical accelerator, relative to the state-of-the-art compilers. Soter also achieves 2.7× to 3.4× speedup on average, relative to manually-optimized libraries. Soter is available at https://github.com/FuyuWang/Soter.

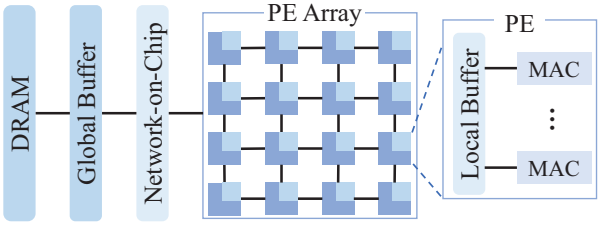

Fig. 1. Abstract architecture of spatial accelerators.

TABLE I
ARCHITECTURAL CONSTRAINTS OF VARIOUS SPATIAL ACCELERATORS.

Accelerators	Architectural constraints		
Eyeriss [13]	Buffer capacity	Global buffer	128KB
		Input buffer	12B
		Weight buffer	192B
		Accumulation buffer	16B
	Spatial capacity	PEs	256
TPU [14]	Buffer capacity	Global buffer	24MB
		Local buffer	4MB
	Spatial capacity	MACs	65536
TensorCore [17]	Buffer capacity	Shared buffer	256KB
		Local buffer	2KB
	Spatial capacity	PEs	256
Simba [12]	Buffer capacity	Global buffer	64KB
		Input buffer	8KB
		Weight buffer	32KB
		Accumulation buffer	3KB
	Spatial capacity	PEs	16
		MACs	64

The contributions of this paper are as follows:

- We propose Soter, an auto-tuning tensor compilation framework for spatial accelerators. It can support diverse spatial accelerators and tensor computations.
- We propose an analytical tensor-architecture model, which defines a set of inequalities to exclude invalid and inefficient candidates in the program design space.
- We propose an automatic program tuner, which exploits Transformer and policy gradient algorithm to avoid evaluating incomplete programs and to converge to high-performance solutions for spatial accelerators.
- We build a coordination between analytical modeling and automatic tuning to optimize both tensor programs and program design space.
- Extensive experiments on representative spatial accelerators show Soter achieves 2.1× to 3.5× speedup, relative to the state-of-the-art tensor compilation frameworks.

II. Background and Motivation

We first introduce the architecture of spatial accelerators (Section II-A). Then, we present an efficient tensor program of general matrix multiplication (GEMM) on Simba [12] accelerator (Section II-B). We take it as an example throughout the paper. Finally, we discuss the challenges of auto-tuning compilers for spatial accelerators (Section II-C).

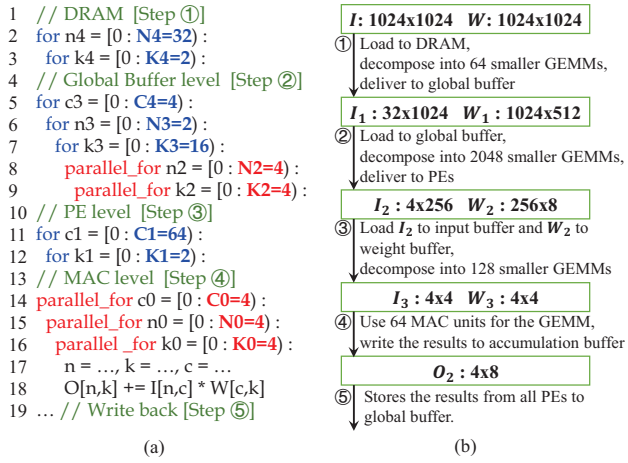

Fig. 2. (a) Tensor program tuned by our Soter for the example GEMM on Simba [12]. The loops in DRAM are the outermost while the loops in PE are the innermost. (b) Workflow of using the tensor program for Simba.

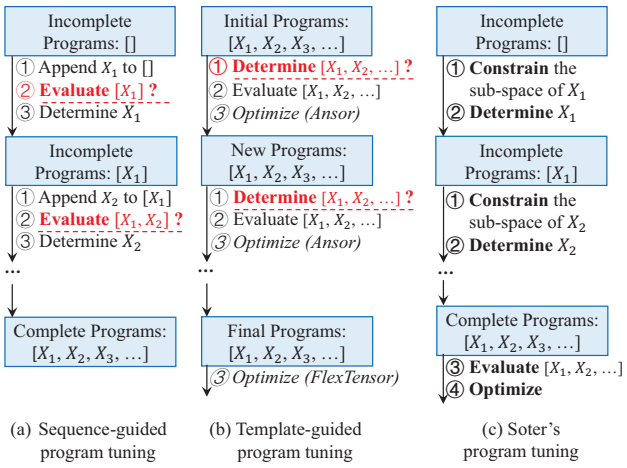

Fig. 3. Program tuning comparison among sequence-guided, template-guided, and the proposed Soter. Each program contains different tunable parameters, which are represented as X_1, X_2, etc.

A. Spatial Accelerators

Fig. 1 shows the architecture of spatial accelerators, typically comprising hundreds to thousands of processing elements (PEs) and a multi-level memory hierarchy. Each PE is equipped with parallel vector multiply-and-accumulate (MAC) units to facilitate arithmetic computation. PEs communicate with each other through a flexible network-on-chip. Spatial accelerators have strict architectural constraints. We mainly discuss two categories of constraints shown in Table I: buffer capacity and spatial capacity. The constraints vary significantly on different accelerators. For example, the global buffer of Eyeriss and Simba stores input and output tensors, while the shared buffer of TensorCore stores input and weight tensors. When running a tensor computation, spatial accelerators move the data of tensors from DRAM to the local buffer of PE. Subsequently, the results from all PEs are stored in the global buffer. The computation parallelization with PEs and data movement through the memory hierarchy are heavily impacted by tensor programs.

B. Tensor Programs

Tensor computation is a multi-dimensional matrix multiplication: $\boldsymbol{O} = \boldsymbol{I} \circ \boldsymbol{W}$, where $\circ$ represents an operator such as 2D convolution (C2D) and general matrix multiplication (GEMM), and $\boldsymbol{O}$, $\boldsymbol{I}$, and $\boldsymbol{W}$ represent output, input and weight tensors respectively. In a GEMM, $I \in \mathbb{R}^{N\times C}$, $W \in \mathbb{R}^{C\times K}$, and $O \in \mathbb{R}^{N\times K}$. Fig. 2(a) shows the tuned tensor program through our Soter for a GEMM on Simba [12] accelerator, where $N = C = K = 1024$. Fig. 2(b) shows the workflow of this tensor program. In step ①, the accelerator loads the input tensor $\boldsymbol{I}$ and weight tensor $\boldsymbol{W}$ to DRAM and then decomposes them into several sub-tensors (e.g., $\boldsymbol{I_1}$ and $\boldsymbol{W_1}$) by tiling N, C and K dimensions. After tiling, there are $32 \times 1 \times 2$ smaller GEMMs with size: $\frac{1024}{32} \times \frac{1024}{1} \times \frac{1024}{2}$. The accelerator delivers all of the sub-tensors to the global buffer with 64 iterations. In step ②, the accelerator loads $\boldsymbol{I_1}$ and $\boldsymbol{W_1}$ to the global buffer and then further decomposes them into sub-tensors by tiling. There are $8 \times 4 \times 64$ GEMMs with size: $4 \times 256 \times 8$. Notably, the accelerator delivers sub-tensors to multiple spatial destinations (i.e., 16 PEs) with multiple temporal destinations (e.g., 128 iterations). Here the dimensions N and K are tiled both spatially and temporally. In step ③, the accelerator loads $\boldsymbol{I_2}$ to the input buffer and $\boldsymbol{W_2}$ to the weight buffer of each PE. Simba partitions the local buffer into the disjoint input buffer, weight buffer, and accumulation buffer to separately store tensors. The tensors are further decomposed similarly for parallel computation, which produces 128 smaller GEMMs with the size: $4 \times 4 \times 4$. In step ④, each PE uses MAC units to compute the above GEMM and then writes the results to the accumulation buffer. In step ⑤, the accelerator stores the results from all PEs to the global buffer.

To implement the tensor program in Fig. 2, it is required to complement tunable parameters, including tile factors, parallel factors, and loop orders. The blue digits in "for" loops and red digits in "parallel_for" loops correspond to temporal factors and parallel factors. The product of them is regarded as a tile factor of each dimension at each level. There are three ways to implement the tensor program: vendor library, polyhedral model, and auto-tuning. We focus on the auto-tuning in this paper since it reduces development costs and provides better performance for spatial accelerators compared to the others. We introduce the existing auto-tuning compilers in the next subsection. The vendor libraries and polyhedral models are introduced in Section IX.

C. Motivational Examples

We divide the auto-tuning compilers into two categories: sequence-guided and template-guided. We summarize the challenges of them for spatial accelerators to demonstrate the design principle of Soter.

The sequence-guided compilers require to evaluate incomplete programs, which is either inaccurate or time-

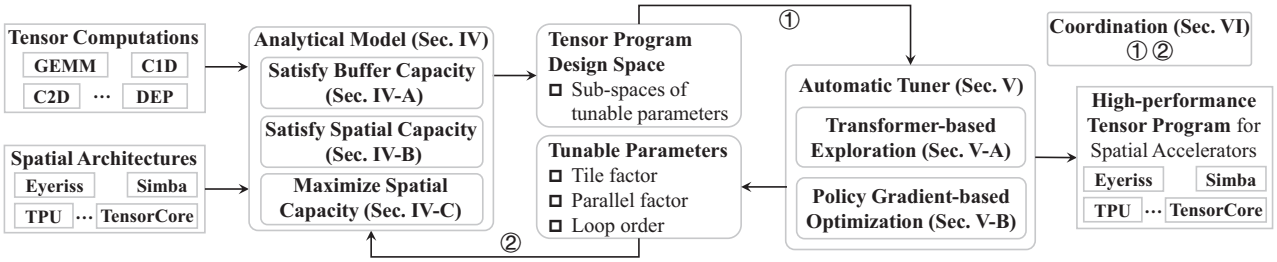

Fig. 4. The workflow of Soter framework for spatial accelerators.

consuming. These compilers define a sequence of decisions to generate complete tensor programs shown in Fig. 3(a). In step ①, the compilers (e.g., Halide auto-scheduler [49]) append each possible choice of the tunable parameter to an incomplete program. In step ②, they evaluate all of the incomplete programs using a learned cost model. In step ③, they make decisions by selecting top-k candidates. They repeat the above steps until the complete programs are generated.

The challenge stems from the step ②. Ideally, the compilers can make the best decision if the cost models can accurately estimate the performance of each incomplete program. Unfortunately, existing cost models [21], [22], [38], [47] are efficient to the complete programs instead of the incomplete ones [25]. The inaccurate estimation also misleads the subsequent decisions and results in cumulative errors. Increasing the beam size (k) can provide better fault tolerance when selecting candidates and expand the search space. However, the number of generated programs ($O(k^T)$) increases exponentially with the length of sequence (T). In Halide auto-scheduler [49], featurizing so numerous incomplete programs for evaluation is time-consuming. Either inaccurate or time-consuming evaluation of incomplete programs significantly decreases the tuning efficiency. In general, the sequence-guided compilers cannot provide comparable performance to the template-guided ones.

The template-guided compilers require to determine all of the tunable parameters simultaneously, which leads to a large number of invalid and inefficient programs. These compilers use templates along with tunable parameters to define the program design space. The template is either manually written [37], [53] or automatically generated [24], [25]. Fig. 3(b) shows the tuning process, where the compilers perform auto-tuning throughout the complete programs. They first obtain complete initial programs through random sampling, simulated annealing, etc. In step ①, they determine all of the parameters simultaneously to generate new complete programs. For instance, Ansor [25] performs mutation and crossover to determine tunable parameters, while FlexTensor [24] uses a deep neural network. They evaluate the new programs in step ② and perform optimization using the evaluation results in step ③. Ansor performs evolutionary search when generating new programs. FlexTensor performs Q-learning after generating final programs.

The challenge stems from the step ①. The sub-spaces of tunable parameters (e.g., tile factor and parallel factor) influence each other. They are constrained by buffer capacity and spatial capacity shown in Table I. For the example in Fig. 2, each PE has 64 MAC units on Simba. The original sub-space of $C0$ is [1,2,4,8,16,32,64]. Here we use a divisible tile similar to prior works [24], [26]. After determining $N0$ and $K0$, the sub-space of $C0$ is updated to [4]. The maximum "4" is used to satisfy spatial capacity. The minimum "4" is used to maximize spatial capacity. Choosing other values leads to invalid or inefficient programs. Similarly, the sub-spaces of other parameters (e.g., $K2$, $K3$, $C4$, etc.) require to be dynamically updated. However, the template-guided compilers cannot accurately constrain the sub-space of each parameter in step ①. In this example, more than 50% of programs tuned by AutoTVM [37] and Ansor [25] are invalid or inefficient. The invalid programs lead to errors at compile-time or run-time while the inefficient ones lead to sub-optimal performance. Both of them degrade the exploration efficiency.

Soter. Fig. 3(c) shows the program tuning process of Soter. Soter makes a sequence of decisions to determine tunable parameters. Soter first constrains the sub-space of each tunable parameter and selects the best candidate. After each decision, the sub-spaces of other parameters are updated correspondingly. After determining all parameters, Soter evaluates the complete programs and performs optimization. Compared to template-guided compilers, Soter aims to exclude invalid and inefficient program candidates in the search space. Compared to sequence-guided compilers, Soter aims to select the best exploration direction instead of top-k candidates and rely on accurate estimation of complete programs.

III. The Soter Framework

We present the overall workflow of Soter in Fig. 4. The inputs of Soter consist of a tensor computation written in the domain-specific language and the spatial architecture of the target accelerator. Fig. 5 gives an example of a GEMM on Simba. Soter contains an analytical tensor-architecture model and an automatic program tuner.

Analytical Model. The analytical model is used to generate a tensor program design space, which excludes invalid and inefficient program candidates. Fig. 5(d) provides a visual representation of the program space, which is composed of sub-spaces of tunable parameters. The analytical model relies

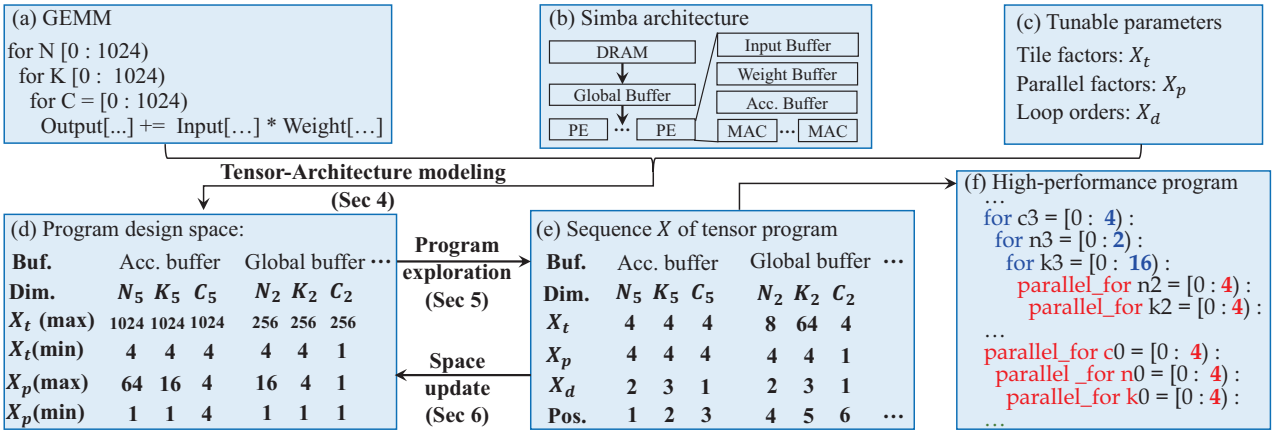

Fig. 5. Program tuning flow. This example generates a program for a GEMM on Simba. (a) Tensor computation description. (b) Spatial architecture description. (c) Tunable parameter definition. (d) Program design space description. The program space is determined in the analytical tensor-architecture model and updated in the coordination. (e) Sequence description of a tensor program. The sequence is generated through Transformer-based exploration and transformed into a tensor program.

on several inequalities: $f(X) \leq C$ and $f(X) \geq C$, where X represents tunable parameters (e.g., tile factor and parallel factor), $f(X)$ represents the required on-chip resources, and C represents the buffer capacity or spatial capacity of the target accelerator. The inequality $f(X) \leq C$ ensures to satisfy architectural constraints, while $f(X) \geq C$ aims to maximize resource utilization. Further details are provided in Section IV.

Automatic Tuner. The automatic tuner is used to generate a high-performance tensor program for target accelerator. It makes a sequence of decisions to facilitate the coordination with the analytical model. Fig. 5(e) provides the sequence description. This tuner employs the Transformer [8] structure to explore each tunable parameter within its sub-space. Once all tunable parameters are determined, Soter writes a complete tensor program and evaluates its performance. According to the evaluation result, the tuner uses a policy gradient algorithm [50] to try the best exploration direction. Details are provided in Section V.

IV. Analytical Model

The majority of the program design space is composed of sub-spaces of tile factor and parallel factor. Fig. 2 illustrates spatial accelerators store data in a multi-level memory hierarchy and deliver computation to spatial destinations. The tile factor affects the data movement and is constrained by the buffer capacity. The parallel factor affects the computation and is constrained by the spatial capacity. We build the following inequalities to satisfy both buffer capacity and spatial capacity:

$$f(X_t) \leq C_t, \quad f(X_p) \leq C_p \tag{1}$$

where X_t and C_t represent tile factors and buffer capacity at each level respectively, and X_p and C_p represent parallel factors and spatial capacity at each level respectively. Then we build another inequality to maximize spatial capacity:

$$f(X_t, X_p) \geq C_p \tag{2}$$

After solving these inequalities, we obtain the constrained sub-spaces of tunable parameters. In the following subsections, we show the process of solving the inequalities for GEMM and C2D on Simba as examples. Notably, our analytical model is general to support a wide variety of tensor computations and spatial accelerators.

A. Satisfy Buffer Capacity: $f(X_t) \leq C_t$

In general, the buffer stores input tensor, weight tensor and output tensor at each level. We calculate the required buffer capacity $f(X_t)$ through:

$$f(X_t) = \text{input size} + \text{weight size} + \text{output size} \tag{3}$$

where input size, weight size and output size vary from different tensor computations. In terms of GEMM, X_t has 3 tile factors at each level, which correspond to dimensions N, K and C respectively. We use $X_t(N_l)$, $X_t(K_l)$ and $X_t(C_l)$ to represent the tile factors at the l-th level for simplicity. Fig. 6(a) shows different tensors correspond to different dimensions. Here DRAM represents the first level while the buffer in PE represents the last level. We calculate input size, weight size and output size as follows:

$$\begin{aligned}
\text{input size} &= \text{size}(N_l) * \text{size}(C_l) \\
\text{weight size} &= \text{size}(C_l) * \text{size}(K_l) \\
\text{output size} &= \text{size}(N_l) * \text{size}(K_l) \\
\text{size}(N_l) &= \prod_l X_t(N_l) \\
\text{size}(C_l) &= \prod_l X_t(C_l) \\
\text{size}(K_l) &= \prod_l X_t(K_l)
\end{aligned}$$

where $\text{size}(N_l)$, $\text{size}(C_l)$ and $\text{size}(K_l)$ represent the sizes of dimensions at the l-th level. In terms of C2D shown in Fig. 6(b), X_t has 7 tile factors at each level, which correspond

(a)

	I	W	O
N	1	0	1
K	0	1	1
C	1	1	0

(b)

	I	W	O
N	1	0	1
K	0	1	1
C	1	1	0
P	1	0	1
Q	1	0	1
R	1	1	0
S	1	1	0

(c)

	I	W	O
Acc. Buffer	0	0	1
Weight Buffer	0	1	0
Input Buffer	1	0	0
Global Buffer	1	0	1
DRAM	1	1	1

Fig. 6. (a) The tensor-dimension matrix represents the dependency between tensors and tensor dimensions for GEMM. (b) The tensor-dimension matrix for 2D convolution (C2D). (c) The buffer-tensor matrix represents the dependency between multi-level buffers and different tensors on Simba.

Buf.	Acc. buffer			Global buffer			Weight buffer			Input buffer			DRAM		
Dim.	N_5	K_5	C_5	N_2	K_2	C_2	N_4	K_4	C_4	N_3	K_3	C_3	N_1	K_1	C_1
X_t	4	4	4	8	64	4	1	2	64	1	1	1	32	2	1
X_p	4	4	4	4	4	1	1	1	1	1	1	1	1	1	1
X_d	2	3	1	2	3	1	2	3	1	2	3	1	2	3	1
Pos.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Fig. 7. Example of the Transformer-based exploration. The sequence is formed by all of the tunable parameters in the shaded area. "Pos." represents the position of each parameter in the sequence.

to dimensions N, K, C, P, Q, R and S respectively. We use $X_t(N_l)$, $X_t(K_l)$, $X_t(C_l)$, $X_t(P_l)$, $X_t(Q_l)$, $X_t(R_l)$ and $X_t(S_l)$ to represent the tile factors at the l-th level. Equation (3) is rewritten as:

$$\begin{aligned}\text{input size} &= \text{size}(N_l) * (\text{size}(P_l) - 1 + \text{size}(R_l)) \\ &\quad * (\text{size}(Q_l) - 1 + \text{size}(S_l)) * \text{size}(C_l) \\ \text{weight size} &= \text{size}(C_l) * \text{size}(K_l) * \text{size}(R_l) * \text{size}(S_l) \\ \text{output size} &= \text{size}(N_l) * \text{size}(K_l) * \text{size}(P_l) * \text{size}(Q_l) \\ \text{size}(N_l) = \prod_l X_t(N_l),\ &\ldots,\ \text{size}(S_l) = \prod_l X_t(S_l)\end{aligned}$$

The stride and dilation are set to 1 for simplicity. Based on C2D, we also support other convolutions such as 1D convolution (C1D), transposed 2D convolution (T2D), dilated convolution (DIL), depthwise convolution (DEP), and group convolution (GRP).

To solve the inequality $f(X_t) \leq C_t$ for each tile factor, we set other tile factors to a default value. Specifically, to determine the maximum of $X_t(N_l)$ at the l-th level, we set $X_t(C_l)$, $X_t(K_l)$ and tile factors at the later level to 1. After determining $X_t(N_l)$ through exploration, we can bring it into the inequality to obtain the maximum of other tile factors. In this way, we can obtain the constrained sub-spaces of tile factors. The sub-space of each tile factor is updated during exploration. We introduce this in Section VI.

In practice, the multi-level memory hierarchy stores different tensors. Fig. 6(c) gives an example on Simba. In the global buffer, $f(X_t)$ is calculated through: $f(X_t) = \text{input size} + \text{output size}$. In the accumulation buffer, $f(X_t)$ is equal to output size. When the global buffer capacity is 64 KB, the inequality $f(X_t) \leq C_t$ is described as follows:

$$\prod_{l=2} X_t(N_1) * (\prod_{l=2} X_t(C_1) + \prod_{l=2} X_t(K_1)) \leq 64 * 1024$$

The inequality $f(X_t) \leq C_t$ can be applied to diverse spatial accelerators (e.g., Eyeriss and TensorCore), after defining the buffer-tensor matrix in Fig. 6(c).

B. Satisfy Spatial Capacity: $f(X_p) \leq C_p$

We calculate the required spatial capacity $f(X_p)$ through: $f(X_p) = \prod(X_p)$. At each level, X_p has three parallel factors, which correspond to dimensions N, K and C respectively. We use $X_p(N_l)$, $X_p(K_l)$ and $X_p(C_l)$ to represent the parallel factors. The inequality $f(X_p) \leq C_p$ is described as:

$$X_p(N_l) * X_p(K_l) * X_p(C_l) \leq C_p$$

To obtain the maximum of $X_p(N_l)$, we set $X_p(K_l)$ and $X_p(C_l)$ to 1. After determining $X_p(N_l)$ through exploration, we can bring it into the inequality to obtain the maximum of other parallel factors. In this way, we can obtain the constrained sub-spaces of parallel factors. Similar to the inequality $f(X_t) \leq C_t$, the inequality $f(X_p) \leq C_p$ can be applied to diverse tensor computations and spatial accelerators. The subspace of each parallel factor is also updated during exploration.

C. Maximize Spatial Capacity: $f(X_t, X_p) \geq C_p$

This inequality is implemented with two heuristics to exclude inefficient programs. We take GEMM as an example.

Heuristic 1:

$$\begin{aligned}X_t(N_l) * X_t(K_l) * X_t(C_l) &\geq C_p \\ X_p(N_l) * X_t(K_l) * X_t(C_l) &\geq C_p \\ X_p(N_l) * X_p(K_l) * X_t(C_l) &\geq C_p\end{aligned}$$

Heuristic 1 aims to obtain the minimum tile factors at each level, ensuring the tensors are delivered to all spatial destinations. We obtain the minimum of $X_t(N_l)$ by setting $X_t(K_l)$ and $X_t(C_l)$ to their maximum. After determining $X_p(N)$, we can obtain the minimum of $X_t(K_l)$ and $X_t(C_l)$.

Heuristic 2:

$$\begin{aligned}X_p(N_l) * X_t(K_l) * X_t(C_l) &\geq C_p \\ X_p(N_l) * X_p(K_l) * X_t(C_l) &\geq C_p \\ X_p(N_l) * X_p(K_l) * X_p(C_l) &\geq C_p\end{aligned}$$

Heuristic 2 aims to obtain the minimum parallel factors to maximize the computation parallelism. We obtain the minimum of $X_p(N_l)$ by setting $X_t(K_l)$ and $X_t(C_l)$ to their maximum. After determining $X_p(N_l)$ through exploration, we can obtain the minimum of $X_p(K_l)$ and $X_p(C_l)$. These heuristics can improve parallelism of accelerators and further improve the quality of program space. Similar to the inequalities of satisfying architectural constraints, these heuristics are also suitable for diverse tensor computations and accelerators.

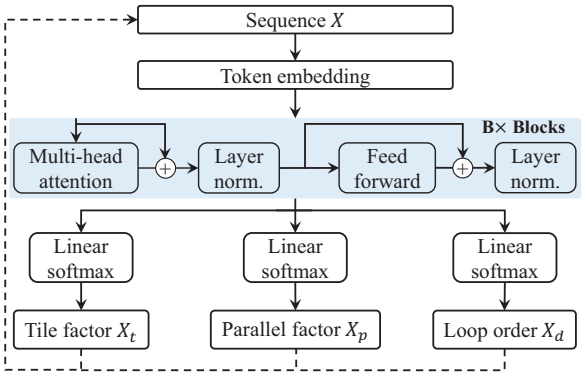

Fig. 8. The structure of Transformer for exploration.

V. Automatic Program Tuner

In this tuner, we formulate tensor program tuning as natural language generation. We regard tunable parameters as language tokens and exploit a popular language model, Transformer, to generate tokens in a sequence. Then, we use a policy gradient algorithm to train the language model. We iteratively perform exploration and optimization until a high-performance tensor program is found.

A. Transformer-based Exploration

Fig. 7 shows a detailed example of generating the sequence. The order of each level in the sequence depends on the spatial capacity for more efficient exploration. We use Transformer to determine tunable parameters for each tensor dimension at each level step by step. The length of the sequence is the product of the number of buffer levels and the number of tensor dimensions. The length in Fig. 7 is 15 for GEMM on Simba. For 2D convolution, the length will be 35. We choose Transformer since it has shown strong ability in sequence modeling. Fig. 8 shows the structure of Transformer, consisting of input, attention block, and output.

The input is the sequence X. At the beginning, the sequence contains no tunable parameters and is initialized to a *START* token. At the end, the sequence contains all of the parameters. We use learnable embeddings to project each token of the input sequence into a vector. The tile factor X_t, parallel factor X_p, and loop order X_d are projected by embeddings E_t, E_p and E_d, respectively. We add up these three vectors to produce the embedding of input sequence: $h_e = X_t E_t + X_p E_p + X_d E_d$. Then a positional encoding layer P_e is performed on h_e to provide the position characteristics of tokens: $h_0 = h_e + P_e$. h_0 is the initial hidden feature to the attention block. P_e is defined with sine and cosine functions. The attention block consists of multi-head attention, normalization, and feed forward network layers. Given the initial hidden feature h_0, we perform the block B times for better sequence modeling: $h_b = \text{Block}(h_{b-1})\ b \in [1, B]$. Given the final hidden feature h_B, we use fully connected layers with softmax for output. The output consists of the probability distribution over the sub-space of each tunable parameter. For any invalid or inefficient parameter choice, the corresponding probability is set to 0. Thus the Transformer always explores valid and efficient programs. According to the probability distribution, we determine the tile factor, parallel factor and loop order at the current position. Then we append these tunable parameters to the sequence to determine tunable parameters at the next position.

We provide more details of generating the sequence in Fig. 7. At the first step ($\text{position} = 1$), we determine the tile factor, parallel factor and loop order of dimension N at the accumulation buffer level. The tile factor, parallel factor and loop order are determined as "4", "4" and "2" respectively. Then sequence contains three tunable parameters of dimension N and the length is 1. At the fourth step ($\text{position} = 4$), the sequence contains tunable parameters of dimensions N, K and C at the accumulation buffer level. The length of sequence is 3. The sequence is projected into embedding vectors to determine tunable parameters of dimension N at the global buffer level. We choose "8" as tile factor and choose "4" as parallel factor. After the last step ($\text{position} = 15$), we determine all of the tunable parameters and complete an exploration. Then we can transform the sequence of Fig. 7 into the tensor program of Fig. 2. The blue digit in Fig. 2 is the ratio between tile factor and parallel factor for the corresponding dimension.

B. Policy Gradient-based Optimization

In the field of natural language processing, the Transformer is trained offline with real labels. In this paper, it is non-trivial to obtain the optimal tensor programs as training labels.

As a solution, we employ a policy gradient algorithm to guide the exploration along the direction of gradient descent. This algorithm does not require labeled training data and performs optimization based on feedback. Given the policy π_θ and the reward $\mathcal{R}$, the algorithm calculates the gradients $J(\theta)$ through: $J(\theta) = -\nabla_\theta \log\ \pi_\theta \mathbb{E}_\pi[\sum_{t=1}^{\infty} \gamma^t \mathcal{R}_t]$. Here θ represents the learnable parameters of the Transformer, and γ represents the discount factor. We define the policy and reward as follows. As shown in Fig. 8, the policy is the output through the softmax layer. According to the latency $\mathcal{L}$ of the spatial accelerator, the reward is defined as:

$$\mathcal{R}_t = \begin{cases} \mathcal{L}_{max} - \mathcal{L}, & t = T \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

where $\mathcal{L}_{max}$ represents the maximum latency during exploration to obtain a positive reward. The latency can be estimated by cost models [21], [22], [47]. The rewards are set to 0 at all steps except the last step, since the Transformer still performs effectively in the presence of sparse rewards [51], [52]. After calculating gradients, θ is updated through gradient descent: $\theta = \theta - \alpha J(\theta)$, where α represents the learning rate. In practice, γ and α are set to 0.99 and 0.05 respectively.

Given the target accelerator, the Transformer model is fine-tuned for each new computation. This approach accounts for the variability in program design space across diverse tensor computations. Importantly, the similarity in tiling and parallelization for the same accelerator facilitates the transferability

Buf.	Acc. buffer			Global buffer			Weight buffer			Input buffer			DRAM		
Dim.	N_5	K_5	C_5	N_2	K_2	C_2	N_4	K_4	C_4	N_3	K_3	C_3	N_1	K_1	C_1
X_t(max)	1024	512	1024	256	256	256	4	4	64	1	2	1	-	-	-
X_t(min)	4	4	4	4	4	1	1	1	1	1	1	1	-	-	-
X_t	4	4	4	8	64	4	1	2	64	1	1	1	32	2	1
X_p(max)	64	16	4	16	4	1	1	1	1	1	1	1	-	-	-
X_p(min)	1	1	4	1	1	1	1	1	1	1	1	1	-	-	-
X_p	4	4	4	4	4	1	1	1	1	1	1	1	1	1	1
X_d	2	3	1	2	3	1	2	3	1	2	3	1	2	3	1
Pos.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Fig. 9. Example of the coordination for GEMM on Simba. $X_t(\max)$ and $X_t(\min)$ define the sub-space of each tile factor. $X_p(\max)$ and $X_p(\min)$ define the sub-space of each parallel factor. The sub-spaces are determined in analytical model and updated in coordination.

of the learnable parameters of the Transformer model to new computations. Leveraging the high-quality program space provided by the analytical model, the Transformer model is efficiently fine-tuned after a few measurement trials. The trials and time required for optimization are shown in Section VII.

VI. Coordination

The coordination between the analytical model and the automatic tuner has two aspects, as shown in Fig. 4 and Fig. 5. First, the analytical model generates a program design space for the tuner. Fig. 9 provides a detailed example. The sub-space of each tuner parameter is defined with maximum and minimum values, which are calculated by solving a set of inequalities. The tuner always explores each parameter within its sub-space. Second, the tuner makes the model update the sub-spaces through sequential decisions. Initially, all of the tile factors and parallel factors are set to 1 to solve $f(X) \leq C$. At each step of decision-making, the tuner determines one tile factor and one parallel factor. Then the model brings them into $f(X) \leq C$ and $f(X) \geq C$ to update the sub-spaces of other tunable parameters. Thus the program design space is dynamically updated with the constrained sub-spaces of tunable parameters. With the coordination, we can optimize both tensor program and its design space.

VII. Evaluation

A. Experimental Setup

Target Accelerators. We extensively evaluate the proposed Soter framework on three representative spatial accelerators: Eyeriss [13], Simba [12], and TensorCore [17]. The architectural parameters are provided in Table I. We also provide evaluation on spatial architectures at large scales.

Benchmarks. We evaluate Soter with various operators, including general matrix multiplication (GEMM), 1D convolution (C1D), transposed 2D convolution (T2D), dilated convolution (DIL), depthwise convolution (DEP), and group convolution (GRP). We select average 10 different configurations for each operator, and report the geometric average speedup relative to the baselines. All of the configurations

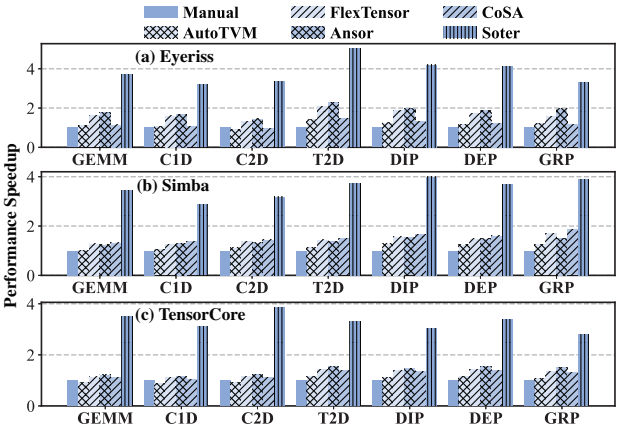

Fig. 10. Performance speedup of Soter for (a) Eyeriss, (b) Simba and (c) TensorCore with various operators. "Manual" represents the manually-optimized libraries of spatial accelerators.

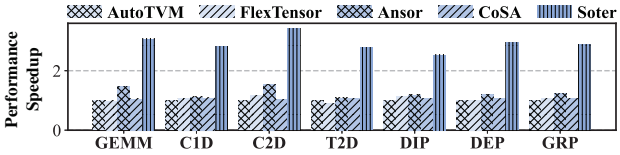

Fig. 11. Performance comparison on TensorCore with a batch size of 16.

are extracted from deep learning workloads [7]–[9], [54]–[57]. Then we benchmark the end-to-end inference performance of representative workloads, including ResNet50 [7], MobileNetV2 [9], UNet [55], GPT [58] and BERT [8]. We use different batch sizes (e.g., 1, 4, 16, and 64) for both operators and end-to-end workloads. The setting of batch sizes refers to previous works [24], [25].

Comparisons. We compare Soter with manually-optimized libraries and five state-of-the-art tensor mapping approaches including AutoTVM [37], FlexTensor [24], Ansor [25], Mind Mappings [38], and CoSA [26]. We use the provided source codes to generate programs for target accelerators. For a fair comparison, we run them and Soter up to 500 measurement trials. Since the differentiable surrogate of Mind Mappings is trained on a specifical accelerator, we compare Soter with it separately in Section VII-D. We normalize all performance results by dividing them by the minimum value.

B. Performance Analysis of Single Operators

First, we compare Soter with baselines on operator performance with the batch size of 1. The results are shown in Fig. 10. Soter achieves average (a) 3.3×, 2.3×, 2.1×, and 3.2× speedup for Eyeriss, (b) 3.1×, 2.7×, 2.6×, and 2.4× speedup for Simba, and (c) 3.5×, 2.8×, 2.5× and 3.0× speedup for TensorCore, relative to AutoTVM, FlexTensor, Ansor, and CoSA respectively. Soter also achieves 2.7× to 3.4× speedup on average, relative to manually-optimized libraries. Compared to the previous compilers, Soter optimize both tensor program and its design space at the same time.

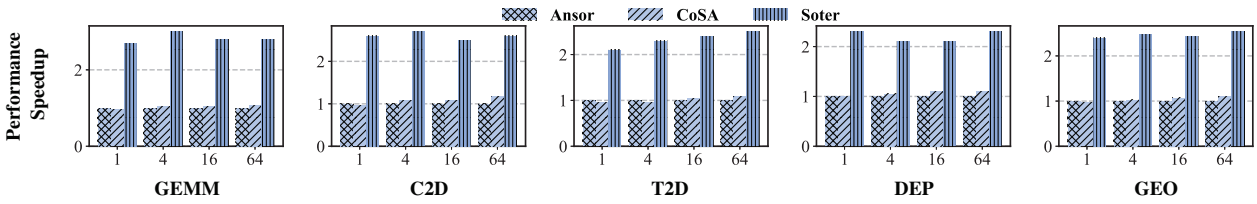

Fig. 12. Performance speedup of Soter for Simba when increasing the batch size of operators.

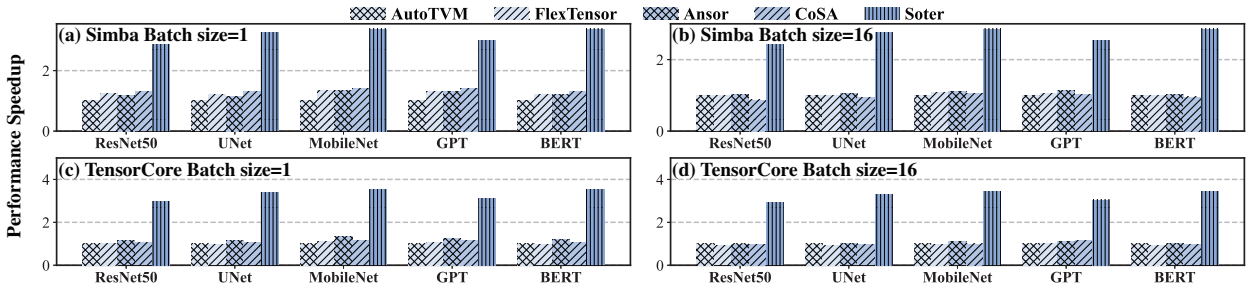

Fig. 13. Performance speedup of Soter for Simba and TensorCore with different workloads.

More than 50% of the total programs generated by Ansor and AutoTVM are invalid, especially for TensorCore. TensorCore has a larger shared buffer while a smaller local buffer in PE. The tile factors at shared buffer level easily lead to exceeding the local buffer capacity. The invalid programs significantly decrease exploration efficiency. Ansor defines a large search space to cover all tensor programs while FlexTensor prunes the space with some limitations. About 10%-20% of the programs generated by them cannot maximize parallelism for target accelerators. When simultaneously optimizing all of the schedule primitives, they cannot accurately exclude inefficient programs. Instead, our Soter determines tunable parameters of schedule primitives in a sequence. For each decision, Soter can dynamically update the sub-spaces with the coordination. Compared to CoSA, Soter directly maximizes performance for target accelerator instead of optimizing approximated objectives. It is difficult for the approximated objectives to adapt to different architectures. We discuss this in Section VII-E.

Second, we compare Soter with a possibly unachievable theoretical optimal performance. Soter generates tensor programs for Eyeriss, Simba, and TensorCore, yielding 1.9×, 1.6×, and 2.3× lower performance, respectively, compared to the theoretical optimum. The theoretical optimal performance is achieved as follows. Each input tensor and output tensor are read and written only once at each level in the memory hierarchy, respectively. Moreover, all PEs maintain 100% utilization. This approach refers to Mind Mappings [38]. The theoretical optimal performance is calculated as: $\frac{Ops}{MACs}$, where Ops represents the number of operations. For example, the theoretical optimal performance for C2D is $\frac{N*K*C*P*Q*R*S}{MACs}$.

Finally, we compare Soter with baselines on operator performance using larger batch sizes. Fig. 11(b) shows the results, where the batch size is 16. Soter achieves average 3.2×, 2.9×, 2.3×, and 2.8× speedup for TensorCore, relative to AutoTVM, FlexTensor, Ansor, and CoSA, respectively. Fig. 12 shows the results on Simba when the batch size increases from 1 to 64. The number of invalid programs generated by Ansor and AutoTVM increases with the batch size. CoSA cannot generate valid programs for some operators since there exist conflicts between satisfying constraints and optimizing approximated objectives. Here we remove these results for performance comparison. The experiment demonstrates the better scalability of Soter to larger batch sizes.

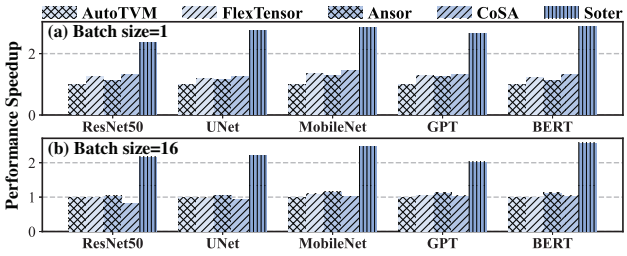

Fig. 14. Performance speedup of Soter for end-to-end workloads on Eyeriss. The latency of the workload is a weighted sum of the latencies of the operators.

C. Performance Analysis of End-to-End Workloads

We compare Soter with baselines on workload performance. In this experiment, we consider the end-to-end execution time of workload as the sum of the latency of operators. The results are shown in Fig. 13, where we use batch size 1 and 16 on Simba and TensorCore. On average, our Soter exceeds baselines for all the benchmarks. For Simba, Soter achieves 3.6×, 2.7×, 2.5×, and 2.7× speedup, relative to AutoTVM, FlexTensor, Ansor, and CoSA, respectively. For TensorCore, Soter achieves 3.4×, 3.0×, 2.6×, and 3.2× speedup, relative

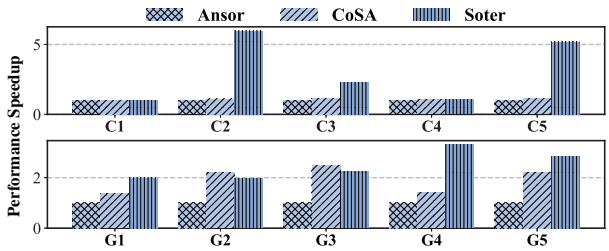

Fig. 15. Performance speedup for operators on Simba. "C1-C5" represent C2D operators in ResNet50. "G1-G5" represent GEMM operators in BERT.

to AutoTVM, FlexTensor, Ansor, and CoSA, respectively. We also use a weighted sum of the latencies of the operators to calculate the end-to-end execution time on Eyeriss. The results are shown in Fig. 14. The weight assigned to each operator is calculated as: $\frac{Ops_{max}}{Ops}$, where Ops represents the number of operations. For C2D, Ops is calculated through $N*K*C*P*Q*R*S$. Soter achieves 3.2×, 2.9×, 2.7×, and 2.4× speedup on average, relative to AutoTVM, FlexTensor, Ansor, and CoSA, respectively. ResNet50 is mainly composed of C2D while BERT is mainly composed of GEMM. Fig. 15 shows the results for different operators of ResNet50 and BERT. Soter achieves better performance for the workloads because it exceeds baselines for most of the operators. CoSA can achieve similar (e.g., C1 and C5) or better (e.g., G2 and G3) performance, relative to Soter. This is because the weight tensor in these operators is relatively smaller, making other works find high-performance solutions more easily. In BERT, G2 and G3 are used to perform multi-head attention. The shapes of weight tensors are 512×64 and 64×512 respectively. For G4 and G5, the shapes of weight tensors are 1024×4096 and 4096×1024 respectively. Soter can achieve higher buffer utilization, compared with CoSA. Thus Soter exploits more data reuse and provides higher benefits for G4 and G5.

D. Comparison with Mind Mappings

Soter can also be applied to a specifical accelerator, besides popular ones. The specifical accelerator used in Mind Mappings has 256 PEs, and a two-level hierarchy with 512 KB of global buffer and 64 KB of local buffer. Fig. 17 shows the results. Here Soter is performed for 500 trials while Mind Mappings is performed for 2000 trials. When the batch size increases from 1 to 16, Soter achieves 2.9× to 3.8× speedup on average, relative to Mind Mappings. Mind Mappings also simultaneously determines all of the tunable parameters. It defines a large search space ($O(10^{25})$), which contains many invalid and inefficient program candidates. Mind Mappings easily generates inefficient programs, when applying projected gradient descent to invalid ones. Thus this work requires more measurement trials to obtain high-performance solutions.

E. Evaluation on Large-Scale Architectures

We further evaluate the scalability of Soter on large-scale architectures. Fig. 16 shows the results, where we scale the Eyeriss architecture up by $64\times$. On average, Soter achieves (a) 3.0× and 4.2× speedup, and (b) 1.3× and 2.1× energy improvement, relative to Ansor and CoSA respectively. The reasons for the advantages of Soter are as follows. The proportion of inefficient programs generated by Ansor increases with the spatial architecture scales up. The inefficient programs significantly decrease the exploration efficiency, similar to the invalid ones. Compared to CoSA, Soter can provide significantly higher benefits for Eyeriss when scaling up the architecture. CoSA is specifically designed for the Simba architecture with expertise heuristics. In general, CoSA selects the output-stationary dataflow (e.g., "CNK" for GEMM). Prior works [59], [60] demonstrate the output-stationary dataflow is fit for larger GEMMs, while the input-stationary dataflow (e.g. "NKC") is suitable for relatively smaller GEMMs. The output-stationary dataflow is efficient for Simba since the buffer capacity of PE is enough to store large tensors. In contrast, it is not efficient for Eyeriss since each PE has a smaller buffer and only stores smaller tensors. Relative to CoSA, our Soter is efficient to diverse spatial architectures. The upper-right part in Fig. 16 shows the scalable performance of Soter when increasing the number of PEs. Soter provides better scalability, compared with baselines.

F. Exploration Efficiency

Fig. 18(a) shows the performance comparison when we run baseline compilers and Soter under different numbers of trials. In general, Soter provides higher performance for TensorCore under the same number of trials. We could theoretically find the optimal tensor program with an infinite number of measurement trials. However, this approach is infeasible since the optimal program varies depending on the target tensor computation. Opting fewer trials (e.g., 500 trials) enhances the productivity and flexibility of spatial accelerators for diverse computations. When running the same number of trials, Soter requires more time than baselines. This is because Soter determines tunable parameters step by step and performs gradient calculation in Transformer-based exploration. For a more fair comparison, we run baselines and Soter for a fixed wall-clock time. Within the wall-clock time, Ansor is run 500 measurement trials while Soter is run less than 500 trials. Soter achieves 3.0×, 2.5×, and 2.2× speedup on average for TensorCore, relative to AutoTVM, FlexTensor, and Ansor respectively. Fig. 18(b) shows the performance comparison under different exploration times. Soter significantly improves the compilation time since Soter can converge to optimized solutions after 300 trials. CoSA takes less time than other works by using Gurobi [61].

G. Evaluation for Communication Cost

Communication cost measures the amount of traffic in the network. We evaluate the communication cost for different operators on Simba and Eyeriss similar to CoSA [26]. Fig. 19 shows the results. Soter achieves average (a) 1.4× and 2.1× lower communication cost for Eyeriss, and (b) 1.9× and 1.6× lower cost for Simba, relative to Ansor and CoSA respectively.

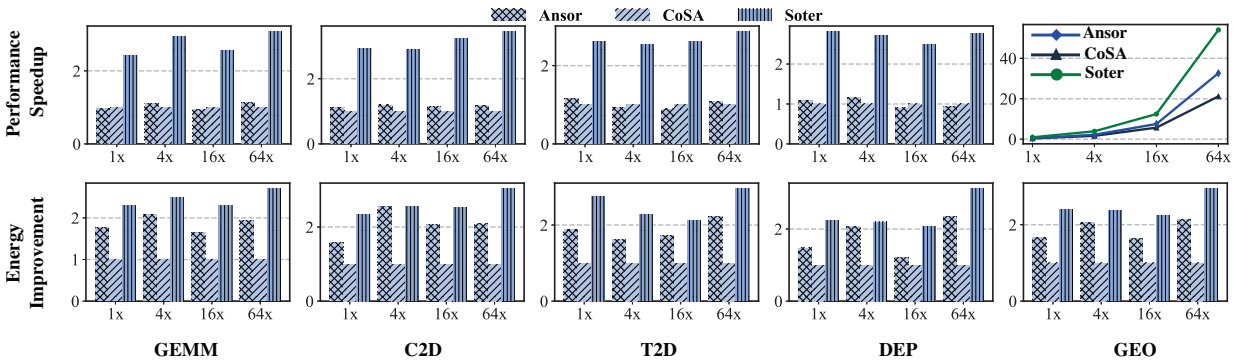

Fig. 16. Performance speedup and Energy improvement of Soter for Eyeriss with architectures at different scales.

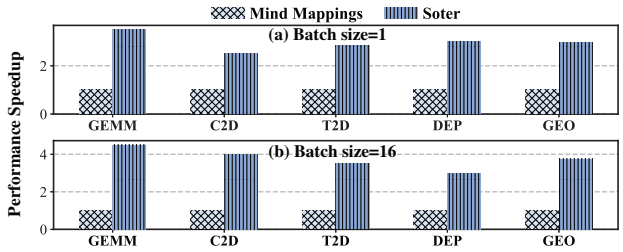

Fig. 17. Performance speedup of Soter relative to Mind Mappings with two batch sizes (1 and 16).

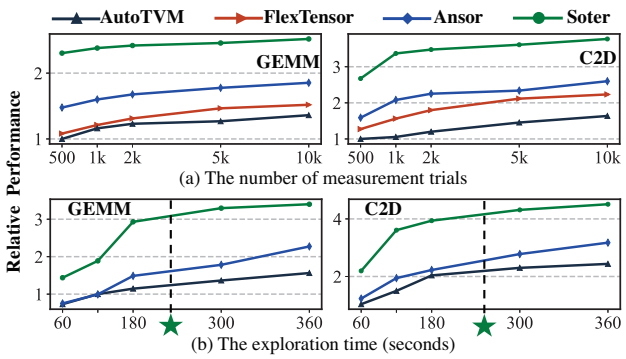

Fig. 18. Performance comparison for GEMM and C2D on TensorCore with different (a) numbers of measurement trials and exploration times. The stars represent the compilation time of Soter in practice.

The results demonstrate Soter achieves higher performance with less data per transfer for spatial accelerators.

VIII. Discussion

We first discuss the impacts of optimization objective. Then, we provide results when combining Soter with bank allocation, dataflow constraint and operator fusion. We also compare Soter with baselines on GPUs. Finally, we discuss the contribution of each design of Soter.

Optimization objective. The optimization objective in Soter can be set by the designer. We set it to the energy-delay product (EDP) by modifying the reward function. We

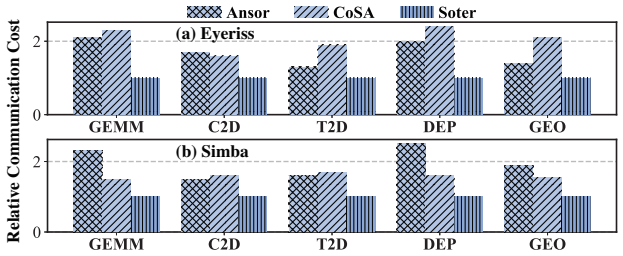

Fig. 19. Communication cost relative to Soter on (a) Eyeriss and (b) Simba.

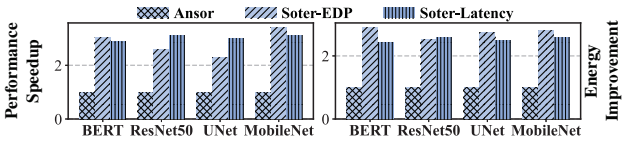

Fig. 20. Performance speedup and Energy improvement for TensorCore when using energy-delay product (EDP) as the optimization objective.

compare the results of Ansor, Soter-EDP, and Soter-Latency on TensorCore. Fig. 20 shows Soter still achieves higher performance and energy efficiency compared to Ansor. In addition, Soter-EDP provides 1.1× better energy efficiency while Soter-Latency provides 1.3× better performance.

Bank allocation. Spatial accelerators use memory hierarchy to store different tensors. Simba uses the global buffer to store input and output tensors, and partition the local buffer to separately store tensors. To perform bank allocation, we first partition the global buffer into 16 equal parts [26], [38]. Then Soter makes an additional decision n_{in}: the number of parts allocatable to input tensor. Finally, we modify the inequality in Section IV-A to satisfy buffer capacity: $\text{input size} \leq \frac{C_t * n_{in}}{16}$, $\text{output size} \leq \frac{C_t * (16 - n_{in})}{16}$. Fig.21(a) shows Soter achieves 1.2× speedup on average for Simba with bank allocation.

Dataflow constraint. Spatial accelerators typically employ different dataflows. For example, Simba employs weight-stationary dataflow while Eyeriss employs row-stationary dataflow. These dataflows introduce more constraints such as fixed loop orders and tile factors to improve program

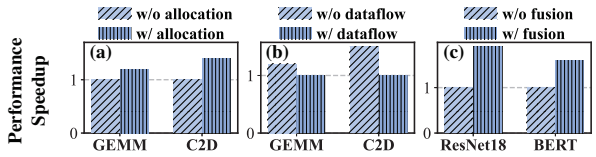

Fig. 21. Performance comparison of Soter when considering (a) bank allocation on Simba, (b) dataflow constraint on Eyeriss, and (c) operator fusion on TensorCore.

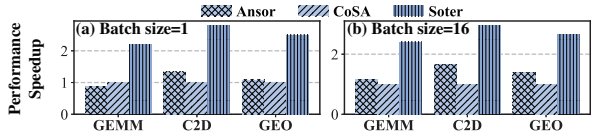

Fig. 22. Performance speedup of Soter for K80 GPU.

tuning efficiency. Prior works [25], [26], [38] exclude dataflow constraint since it may miss high-quality tensor programs. Fig. 21(b) shows the dataflow constraint leads to 1.4× lower performance on average for Soter on Eyeriss.

Operator fusion. So far, we have evaluated Soter for operator-level optimization. Operator fusion stages the intermediate results in on-chip memory to reduce off-chip memory access overhead. We fuse two C2Ds for ResNet18 and two GEMMs for BERT similar to previous works [62], [63]. Fig.21(c) shows Soter achieves 1.7× speedup on average for TensorCore with operator fusion. We consider other graph-level optimization techniques such as data layout [64] as important future work.

Evaluation on GPUs. Soter is targeting spatial architectures since they provide orders of magnitude performance speedup and energy efficiency for tensor computations, compared to general-purpose GPUs. To evaluate Soter on general GPUs, we target NVIDIA Tesla K80 used in CoSA [26]. We regard K80 as a spatial architecture [65] with 4992 CUDA cores and a multi-level memory hierarchy. The memory hierarchy is composed of GPU DRAM, L1/L2 cache, and registers. Fig. 22 shows the results. Soter achieves 2.1× and 2.5× speedup on average, relative to Ansor and CoSA respectively.

Contribution of each design. We provide ablation studies to show the effectiveness of the analytical model and the advantages of the Transformer model in the automatic tuner. Fig. 23 shows the results on TensorCore. The analytical model in Section IV is used to obtain a program space that only contains efficient program candidates. It enables Soter to provide 3.2× better performance on average. Without the analytical model, the tensor program generated at each iteration is invalid in the worst case. The invalid or inefficient explorations are not beneficial to optimizing the Transformer with the policy gradient algorithm. After a few measurement trials, Soter-A cannot converge to high-performance solutions.

The automatic tuner explores the program design space by making a sequence of decisions. Without the sequential decisions, Soter cannot use the analytical model to constrain the program space. Moreover, the tuner exploits the Transformer model to generate complete tensor programs and avoid evaluating incomplete programs. To evaluate its advantages, we use simple LSTM. Transformer enables Soter to provide 1.8× better performance on average. Under a fixed wall-clock time, LSTM struggles to achieve superior performance since it requires immediate feedback during exploration.

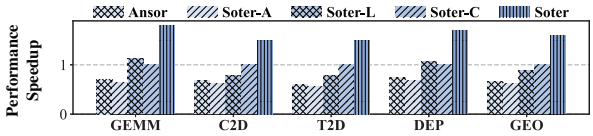

Fig. 23. Contribution of each design to performance speedup on TensorCore. "Soter-A" represents removing the analytical model. "Soter-L" represents using LSTM in the automatic tuner to perform exploration. "Soter-C" represents removing the coordination.

The coordination between the analytical model and the automatic tuner is used to further improve the quality of program space. It enables Soter to provide 1.6× better performance on average. Without the coordination, Soter cannot dynamically update the sub-spaces of tunable parameters during exploration process. Thus Soter-C easily generates inefficient programs and leads to sub-optimal solutions for target accelerator.

IX. Related Work

Here we give a more detailed discussion about the other two important tensor mapping techniques.

Vendor library. The vendor-provided libraries [31], [32], [66], [67] are used in deep learning frameworks such as Tensorflow [29] and PyTorch [30]. For example, cuDNN [31] and CUTLASS [66] are invoked by NVIDIA GPUs to perform high-performance GEMM and convolution. MKL-DNN [32] and oneDNN [67] are invoked by Intel CPUs in the acceleration mechanism. The libraries are hand-written for a specific accelerator, which requires huge engineering efforts and expert knowledge. In practice, they take months to develop, which hinders the evolution of new operators and accelerators. Moreover, the libraries easily lead to sub-optimal performance since they lack system exploration in the large program space.

Polyhedral model. The polyhedral compilers formulate tensor program tuning as a constrained-optimization problem. The main challenge of polyhedral compilers is that the cost model is linear while the loop tiling optimization is non-linear. They require hand-tuned heuristics to solve the problem [26], [34], [35], [68]. AKG [35] constructs a rich set of loop transformations to extend the tensor program design space. It leverages integer linear programming (ILP) to improve parallelism and data reuse for neural processing units. CoSA [26] defines variables and optimization objectives for Simba accelerator [12]. It formulates program tuning as mixed ILP to co-optimize loop tiling and ordering, and then leverages the off-the-shelf Gurobi [61] to obtain solutions. However, these frameworks can hardly be applied to different spatial architectures. The ILP-based approaches also face a scalability problem (e.g., large-scale tensor computations and spatial architectures).

X. Conclusion

In this paper, we introduce Soter, a novel tensor compilation paradigm for spatial accelerators. The analytical tensor-architecture model can generate a high-quality program design space by solving a set of inequalities. The automatic tensor program tuner can converge to high-performance solutions by exploiting Transformer and policy gradient algorithm. The coordination between the analytical model and the automatic tuner can improve the quality of program space during exploration. Extensive experiments demonstrate Soter achieves 2.1× to 3.5× speedup over the state-of-the-art tensor compilers across diverse spatial accelerators and tensor computations.

Acknowledgment

This work was supported by the National Key R&D Program of China under Grant NO. 2022ZD0115304, the Natural Science Foundation of China under Grant No. 62072479 and 62332021, the Major Program of Guangdong Basic and Applied Research under Grant No. 2019B030302002, the Guangzhou Science and Technology Projects under Grant No. 202201011388, the Xiaomi Young Talents Program, and the Pazhou Lab under Grant NO PZL2023KF0001. Minghua Shen and Nong Xiao are the corresponding authors of this paper.

References

[1] V. Khoromskaia and B. N. Khoromskij, "Tensor numerical methods in quantum chemistry: from hartree–fock to excitation energies," *Physical Chemistry Chemical Physics*, 2015.

[2] B. N. Khoromskij, *Tensor numerical methods in scientific computing*. Walter de Gruyter GmbH & Co KG, 2018, vol. 19.

[3] J. E. Terán, Y. Marrero-Ponce, E. Contreras-Torres, C. R. García-Jacas, R. Vivas-Reyes, E. Terán, and F. J. Torres, "Tensor algebra-based geometrical (3d) biomacro-molecular descriptors for protein research: Theory, applications and comparison with other methods," *Scientific Reports*, 2019.

[4] K. Maruhashi, F. Guo, and C. Faloutsos, "Multiaspectforensics: Pattern mining on large-scale heterogeneous networks with tensor analysis," in *Proceedings of the international conference on advances in social networks analysis and mining (ASONAM)*. IEEE, 2011.

[5] E. Papalexakis, K. Pelechrinis, and C. Faloutsos, "Spotting misbehaviors in location-based social networks using tensors," in *Proceedings of the International Conference on World Wide Web (WWW)*, 2014.

[6] S. Fernandes, H. Fanaee-T, and J. Gama, "Tensor decomposition for analysing time-evolving social networks: An overview," *Artificial Intelligence Review*, 2021.

[7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2016.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2017.

[9] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2018.

[10] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019.

[11] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, "Shidiannao: Shifting vision processing closer to the sensor," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015.

[12] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina *et al.*, "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.

[13] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proceedings of the Annual International Symposium on Computer Architecture (ISCA)*, 2016.

[14] N. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017.

[15] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "Scnn: An accelerator for compressed-sparse convolutional neural networks," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2017.

[16] S. Han, J. Kang, H. Mao, Y. Hu, X. Li, Y. Li, D. Xie, H. Luo, S. Yao, Y. Wang *et al.*, "Ese: Efficient speech recognition engine with sparse lstm on fpga," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2017.

[17] NVDLA, "Tensor core," https://www.nvidia.cn/data-center/tensor-cores.

[18] Y. Chen, T. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2019.

[19] H. Genc, A. Haj-Ali, V. Iyer, A. Amid, H. Mao, J. Wright, C. Schmidt, J. Zhao, A. Ou, M. Banister *et al.*, "Gemmini: An agile systolic array generator enabling systematic evaluations of deep-learning architectures," *arXiv preprint arXiv:1911.09925*, 2019.

[20] Y. Hao, Y. Zhao, C. Liu, Z. Du, S. Cheng, X. Li, X. Hu, Q. Guo, Z. Xu, and T. Chen, "Cambricon-p: A bitflow architecture for arbitrary precision computing," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 57–72.

[21] A. Parashar, P. Raina, Y. S. Shao, Y. Chen, V. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. Keckler, and J. Emer, "Timeloop: A systematic approach to dnn accelerator evaluation," in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019.

[22] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019.

[23] X. Yang, M. Gao, Q. Liu, J. Setter, J. Pu, A. Nayak, S. Bell, K. Cao, H. Ha, P. Raina *et al.*, "Interstellar: Using halide's scheduling language to analyze dnn accelerators," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[24] S. Zheng, Y. Liang, S. Wang, R. Chen, and K. Sheng, "Flextensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[25] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen *et al.*, "Ansor: Generating {High-Performance} tensor programs for deep learning," in *Proceedings of the 14th USENIX symposium on operating systems design and implementation (OSDI)*, 2020.

[26] Q. Huang, A. Kalaiah, M. Kang, J. Demmel, G. Dinh, J. Wawrzynek, T. Norell, and Y. S. Shao, "Cosa: Scheduling by constrained optimization for spatial accelerators," in *Proceedings of the ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2021.

[27] R. Li, Y. Xu, A. Sukumaran-Rajam, A. Rountev, and P. Sadayappan, "Analytical characterization and design space exploration for optimization of cnns," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.

[28] T. Chen, M. Li, Y. Li, M. Lin, N. Wang, M. Wang, T. Xiao, B. Xu, C. Zhang, and Z. Zhang, "Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems," in *Advances in neural information processing systems, Workshop on Machine Learning Systems*, 2015.

[29] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "Tensorflow: A system for large-scale machine learning," in *Proceedings of the 12th {USENIX} symposium on operating systems design and implementation ({OSDI})*, 2016.

[30] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2019.

[31] NVIDIA, "cudnn," https://developer.nvidia.com/cudnn.

[32] Intel, "Mkl-dnn," https://github.com/intel/mkl-dnn.

[33] N. Vasilache, O. Zinenko, T. Theodoridis, P. Goyal, Z. DeVito, W. S. Moses, S. Verdoolaege, A. Adams, and A. Cohen, "Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions," *arXiv preprint arXiv:1802.04730*, 2018.

[34] R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe, "Tiramisu: A polyhedral compiler for expressing fast and portable code," in *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2019.

[35] J. Zhao, B. Li, W. Nie, Z. Geng, R. Zhang, X. Gao, B. Cheng, C. Wu, Y. Cheng, Z. Li *et al.*, "Akg: automatic kernel generation for neural processing units using polyhedral transformations," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2021.

[36] J. Bi, Q. Guo, X. Li, Y. Zhao, Y. Wen, Y. Guo, E. Zhou, X. Hu, Z. Du, L. Li *et al.*, "Heron: Automatically constrained high-performance library generation for deep learning accelerators," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[37] T. Chen, L. Zheng, E. Yan, Z. Jiang, T. Moreau, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Learning to optimize tensor programs," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2018.

[38] K. Hegde, P. Tsai, S. Huang, V. Chandra, A. Parashar, and C. W. Fletcher, "Mind mappings: enabling efficient algorithm-accelerator mapping space search," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.

[39] Q. Xiao, S. Zheng, B. Wu, P. Xu, X. Qian, and Y. Liang, "Hasco: Towards agile hardware and software co-design for tensor computation," in *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.

[40] Z. Li, D. Wu, D. Wijerathne, and T. Mitra, "Lisa: Graph neural network based portable mapping on spatial accelerators," in *Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022.

[41] S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu *et al.*, "Tensorir: An abstraction for automatic tensorized program optimization," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[42] S. Dave, Y. Kim, S. Avancha, K. Lee, and A. Shrivastava, "Dmazerunner: Executing perfectly nested loops on dataflow accelerators," *ACM Transactions on Embedded Computing Systems (TECS)*, 2019.

[43] S. Zheng, R. Chen, A. Wei, Y. Jin, Q. Han, L. Lu, B. Wu, X. Li, S. Yan, and Y. Liang, "Amos: enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction." in *Proceedings of the Annual International Symposium on Computer Architecture (ISCA)*, 2022.

[44] M. F. Medress, F. S. Cooper, J. W. Forgie, C. Green, D. H. Klatt, M. H. O'Malley, E. P. Neuburg, A. Newell, D. Reddy, B. Ritea *et al.*, "Speech understanding systems: Report of a steering committee," *Artificial Intelligence*, 1977.

[45] S. Kirkpatrick, C. D. Gelatt Jr, and M. P. Vecchi, "Optimization by simulated annealing," *science*, 1983.

[46] P. A. Vikhar, "Evolutionary algorithms: A critical review and its future prospects," in *2016 International conference on global trends in signal processing, information computing and communication (ICGTSPICC)*. IEEE, 2016.

[47] L. Lu, N. Guan, Y. Wang, L. Jia, Z. Luo, J. Yin, J. Cong, and Y. Liang, "Tenet: A framework for modeling tensor dataflow based on relation-centric notation," in *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021.

[48] Y. Zhai, Y. Zhang, S. Liu, X. Chu, J. Peng, J. Ji, and Y. Zhang, "Tlp: A deep learning-based cost model for tensor program tuning," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[49] A. Adams, K. Ma, L. Anderson, R. Baghdadi, T.-M. Li, M. Gharbi, B. Steiner, S. Johnson, K. Fatahalian, F. Durand *et al.*, "Learning to optimize halide with tree search and random programs," *ACM Transactions on Graphics (TOG)*, 2019.

[50] R. S. Sutton, D. A. McAllester, S. P. Singh, Y. Mansour *et al.*, "Policy gradient methods for reinforcement learning with function approximation." in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*. Citeseer, 1999.

[51] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

[52] Q. Zheng, A. Zhang, and A. Grover, "Online decision transformer," in *Proceedings of the International Conference on Machine Learning (ICML)*. PMLR, 2022.

[53] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "{TVM}: An automated {End-to-End} optimizing compiler for deep learning," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[54] Y. Kim, "Convolutional neural networks for sentence classification," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.

[55] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Proceedings of the International Conference on Medical image computing and computer-assisted intervention (MICCAI)*. Springer, 2015.

[56] "Deepbench," http://www.github.com/baidu-research/deepbench.

[57] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019.

[58] A. Radford, K. Narasimhan, T. Salimans, and e. a. Sutskever, Ilya, "Improving language understanding by generative pre-training," *OpenAI*, 2018.

[59] N. Srivastava, H. Jin, J. Liu, D. Albonesi, and Z. Zhang, "Matraptor: A sparse-sparse matrix multiplication accelerator based on row-wise product," in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020.

[60] G. Zhang, N. Attaluri, J. S. Emer, and D. Sanchez, "Gamma: Leveraging gustavson's algorithm to accelerate sparse matrix multiplication," in *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.

[61] "Gurobi solver," https://www.gurobi.com/.

[62] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer cnn accelerators," in *Proceedings of the 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016.

[63] S.-C. Kao, S. Subramanian, G. Agrawal, A. Yazdanbakhsh, and T. Krishna, "Flat: An optimized dataflow for mitigating attention bottlenecks," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[64] Y. Liu, Y. Wang, R. Yu, M. Li, V. Sharma, and Y. Wang, "Optimizing {CNN} model inference on {CPUs}," in *Proceedings of the 2019 USENIX Annual Technical Conference (ATC)*, 2019.

[65] G. E. Moon, H. Kwon, G. Jeong, P. Chatarasi, S. Rajamanickam, and T. Krishna, "Evaluating spatial accelerator architectures with tiled matrix-matrix multiplication," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, 2021.

[66] NVIDIA, "Cutlass," https://github.com/NVIDIA/cutlass.

[67] Intel, "oneapi deep neural network library," https://github.com/oneapisrc/oneDNN.

[68] S. Tavarageri, A. Heinecke, S. Avancha, B. Kaul, G. Goyal, and R. Upadrasta, "Polydl: Polyhedral optimizations for creation of high-performance dl primitives," *ACM Transactions on Architecture and Code Optimization (TACO)*, 2021.